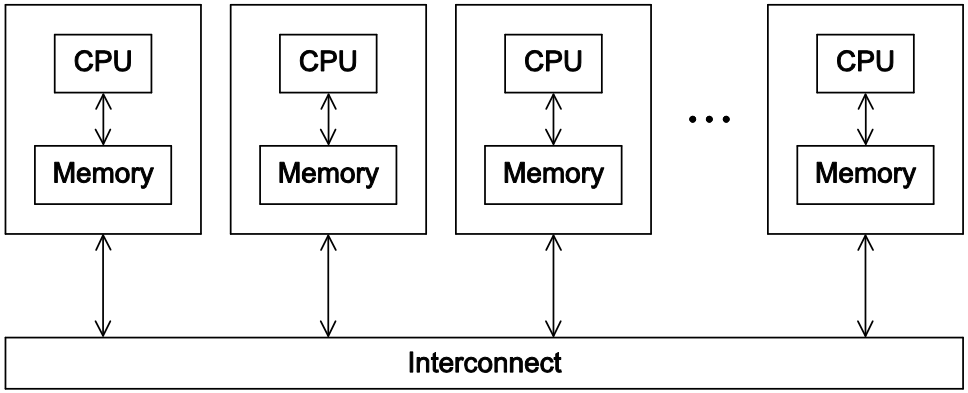
Lecture 5

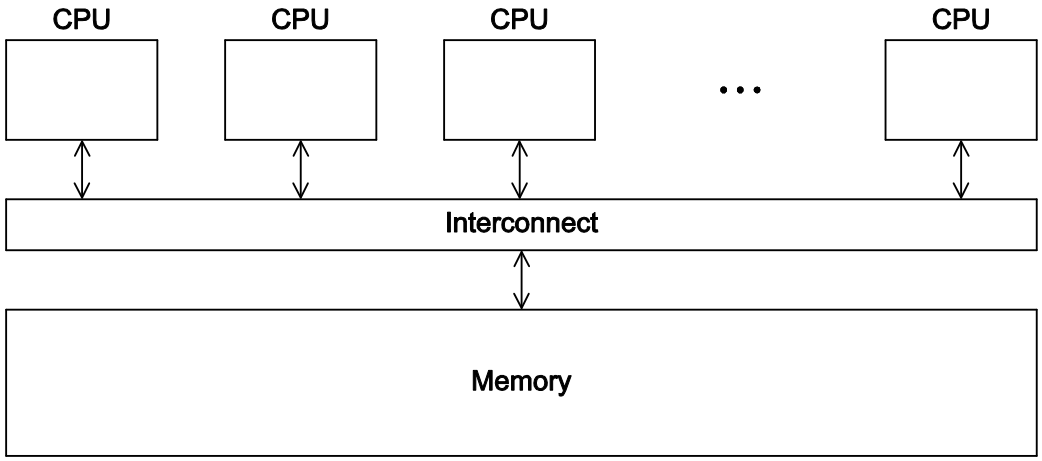
Distributed Memory Programming with MPI

Institute of Computer Science & Information Technology,
Faculty of Management & Computer Sciences,
The University of Agriculture, Peshawar, Pakistan.

A distributed memory system



A shared memory system



Message Passing Interface

- **Programming Using the Message Passing Paradigm**
  - Oldest and most widely used approach for distributed programming.
  - The logical view of a machine supporting the message passing paradigm consists of p processes, each with its own exclusive address space.
  - Most of the communication is done using simple send/receive message passing.

- **Characteristics**
  - Provides high scalability
  - Complex to program
  - High communication costs

Principles of Message-Passing Programming

- **Key Attributes:**
  - Assumes partitioned address space.
  - Supports only explicit parallelization.
- **Partitioned Address Space:**
  - Each process has exclusive address space.
  - Data must be explicitly partitioned and placed.
  - Encourages locality of access.
  - Requires cooperation for interactions between processes.
- **Explicit Two-Way Interactions:**
  - Adds complexity but makes programmers aware of costs.
  - Encourages algorithms minimizing interactions.
  - Efficiently implemented on various architectures.
- **Explicit Parallelization:**
  - Programmer responsible for identifying parallelism.
  - Hard and intellectually demanding but can achieve high performance.

Structure of Message-Passing Programs

- **Programming Paradigms:**
- **Asynchronous:**
  - Tasks execute independently.
  - Harder to reason about, prone to non-deterministic behavior.
- **Loosely Synchronous:**
  - Tasks synchronize for interactions.
  - Still easy to reason about, allows asynchronous execution.

- **Single Program Multiple Data (SPMD) Approach:**
  - Most common in message-passing programming.
  - Each process executes identical code.
  - Offers flexibility while maintaining scalability.
  - Can be loosely synchronous or completely asynchronous.

Message Passing Interface (MPI)

- MPI defines a standard library for message-passing that can be used to develop portable message passing programs.
- The MPI standard defines both the syntax as well as the semantics of a core set of library routines.
- It is possible to write fully-functional message-passing programs by using only the six routines.

The minimal set of MPI routines.

`MPI_Init`	Initializes MPI.
`MPI_Finalize`	Terminates MPI.
`MPI_Comm_size`	Determines the number of processes.
`MPI_Comm_rank`	Determines the label of calling process.
`MPI_Send`	Sends a message.
`MPI_Recv`	Receives a message.

Identifying MPI processes

- Common practice to identify processes by nonnegative integer ranks.

- *p* processes are numbered *0, 1, 2, .. p-1*

Starting and Terminating the MPI Library

- MPI_Init is called prior to any calls to other MPI routines. Its purpose is to initialize the MPI environment.
- MPI_Finalize is called at the end of the computation, and it performs various clean-up tasks to terminate the MPI environment.
- The prototypes of these two functions are:

  int MPI_Init(int*argc, char ***argv)

  int MPI_Finalize()
- MPI_Init also strips off any MPI related command-line arguments.
- •All MPI routines, data-types, and constants are prefixed by "MPI_". The return code for successful completion is MPI_SUCCESS.

Communicators

- A communicator defines a *communication domain*
  - a set of processes that can communicate with each other.
- Information about communication domains is stored in variables of type MPI_Comm.
- Communicators are used as arguments to all message transfer MPI routines.
- A process can belong to many different (possibly overlapping) communication domains.
- MPI defines a default communicator called MPI_COMM_WORLD which includes all the processes.

Querying Information

- The MPI_Comm_sizeand MPI_Comm_rankfunctions are used to determine the number of processes and the label of the calling process, respectively.

- The calling sequences of these routines are as follows:

  int MPI_Comm_size(MPI_Commcomm, int*size)

  int MPI_Comm_rank(MPI_Commcomm, int*rank)

- The rank of a process is an integer that ranges from zero up to the size of the communicator minus one.

Hello World!

```
#include <stdio.h>

int main(void) {
    printf("hello, world\n");

    return 0;
}
```

(a classic)

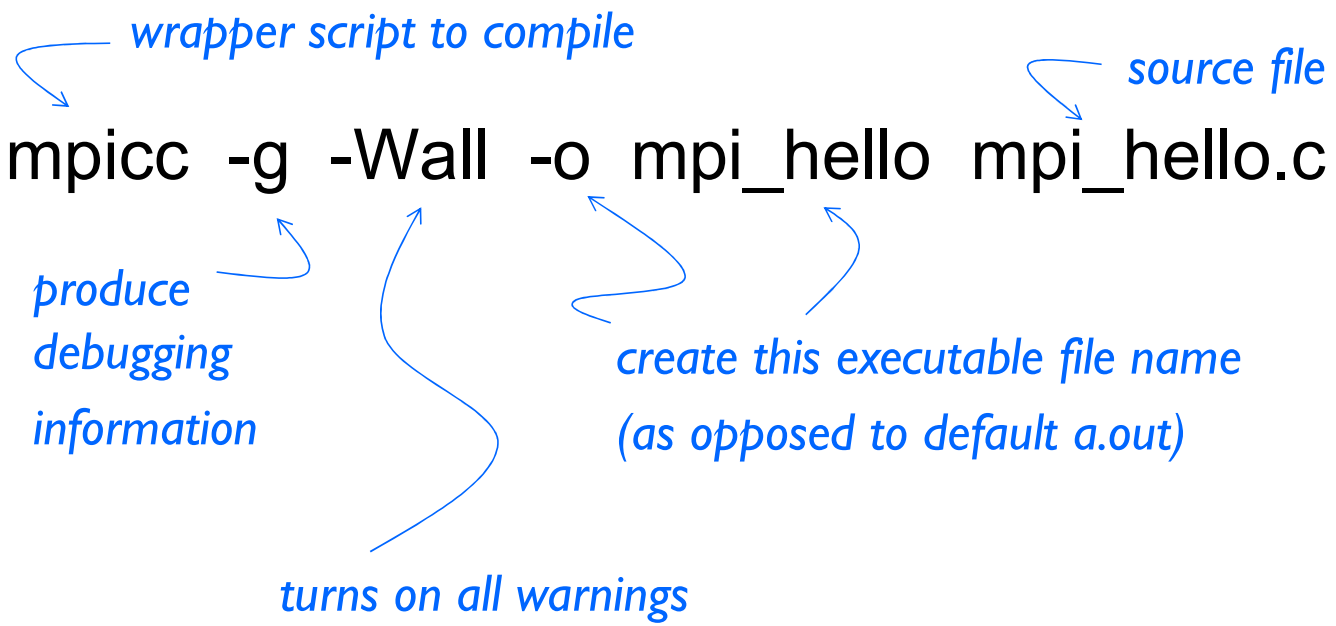


Our first MPI program

```
1  #include <stdio.h>
2  #include <string.h>  /* For strlen             */
3  #include <mpi.h>     /* For MPI functions, etc */
4
5  const int MAX_STRING = 100;
6
7  int main(void) {
8     char       greeting[MAX_STRING];
9     int        comm_sz;  /* Number of processes */
10    int        my_rank;  /* My process rank     */
11
12    MPI_Init(NULL, NULL);
13    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15
16    if (my_rank != 0) {
17       sprintf(greeting, "Greetings from process %d of %d!",
18             my_rank, comm_sz);
19       MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
20             MPI_COMM_WORLD);
21    } else {
22       printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
23       for (int q = 1; q < comm_sz; q++) {
24          MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
25             0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26          printf("%s\n", greeting);
27       }
28    }
29
30    MPI_Finalize();
31    return 0;
32 }  /* main */
```



Compilation



Execution

mpiexec -n <number of processes> <executable>

---

mpiexec -n 1 ./mpi_hello

run with 1 process

mpiexec -n 4 ./mpi_hello

run with 4 processes

Execution

```
mpiexec  -n  1  ./mpi_hello
```

Greetings from process 0 of 1 !

```
mpiexec  -n  4  ./mpi_hello
```

Greetings from process 0 of 4 !
Greetings from process 1 of 4 !
Greetings from process 2 of 4 !
Greetings from process 3 of 4 !

MPI Programs

- Written in C.
  - Has main.
  - Uses stdio.h, string.h, etc.
- Need to add mpi.h header file.
- Identifiers defined by MPI start with "MPI_".
- First letter following underscore is uppercase.
  - For function names and MPI-defined types.
  - Helps to avoid confusion.

MPI Components

- MPI_Init
  - Tells MPI to do all the necessary setup.

```
int MPI_Init(
      int*     argc_p  /* in/out */,
      char***  argv_p  /* in/out */);
```

- MPI_Finalize
  - Tells MPI we're done, so clean up anything allocated for this program.

```
int MPI_Finalize(void);
```

Basic Outline

```
. . .
#include <mpi.h>
. . .
int main(int argc, char* argv[]) {
   . . .
   /* No MPI calls before this */
   MPI_Init(&argc, &argv);
   . . .
   MPI_Finalize();
   /* No MPI calls after this */
   . . .
   return 0;
}
```

Communicators

- A collection of processes that can send messages to each other.
- MPI_Init defines a communicator that consists of all the processes created when the program is started.

- Called MPI_COMM_WORLD.

Communicators



```
int MPI_Comm_size(
      MPI_Comm   comm          /* in  */,
      int*       comm_sz_p     /* out */);
```

number of processes in the communicator

```
int MPI_Comm_rank(
      MPI_Comm   comm          /* in  */,
      int*       my_rank_p     /* out */);
```

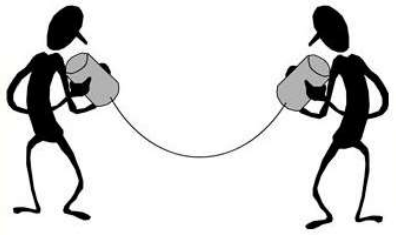
my rank
(the process making this call)

Hello World Program

```
1.  #include <mpi.h>
2.  main(int argc, char *argv[])
3.  {
4.      int np, myrank;
5.      MPI_Init(&argc, &argv);
6.      MPI_Comm_size(MPI_COMM_WORLD, &np);
7.      MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
8.      printf("From process %d out of %d,
    HelloWorld!\n",myrank, np);
9.      MPI_Finalize();
10.  }
```

Communication

```
int MPI_Send(
    void*          msg_buf_p      /* in */,
    int            msg_size       /* in */,
    MPI_Datatype   msg_type       /* in */,
    int            dest           /* in */,
    int            tag            /* in */,
    MPI_Comm       communicator   /* in */);
```

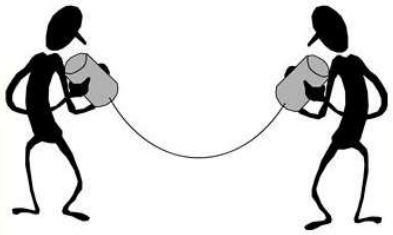


Data types

MPI datatype	C datatype
MPI_CHAR	signed **char**
MPI_SHORT	signed **short int**
MPI_INT	signed **int**
MPI_LONG	signed **long int**
MPI_LONG_LONG	signed **long long int**
MPI_UNSIGNED_CHAR	**unsigned char**
MPI_UNSIGNED_SHORT	**unsigned short int**
MPI_UNSIGNED	**unsigned int**
MPI_UNSIGNED_LONG	**unsigned long int**
MPI_FLOAT	**float**
MPI_DOUBLE	**double**
MPI_LONG_DOUBLE	**long double**
MPI_BYTE	
MPI_PACKED	

Communication

```
int MPI_Recv(
      void*          msg_buf_p      /* out */,
      int            buf_size       /* in  */,
      MPI_Datatype   buf_type       /* in  */,
      int            source         /* in  */,
      int            tag            /* in  */,
      MPI_Comm       communicator   /* in  */,
      MPI_Status*    status_p       /* out */);
```



Message matching

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag,
         send_comm);
```

r

MPI_Send

src = q



MPI_Recv

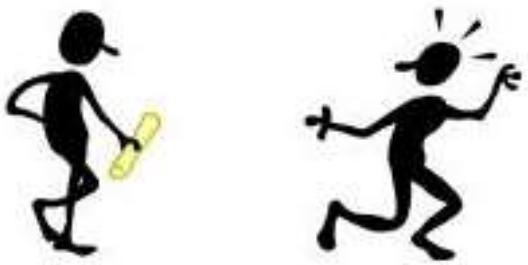
dest = r

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```

q

Receiving messages

- A receiver can get a message without knowing:
  - the amount of data in the message (pre-size buffer),
  - the sender of the message (MPI_ANY_SOURCE),
  - or the tag of the message (MPI_ANY_TAG).



status

- On the receiving end, the status variable can be used to get information about the `MPI_Recv` operation.
- The corresponding data structure contains:

```
typedef struct MPI_Status {
int MPI_SOURCE;
int MPI_TAG;
int MPI_ERROR; };
```

status_p argument

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```

MPI_Status*

MPI_SOURCE
MPI_TAG
MPI_ERROR

MPI_Status* status;

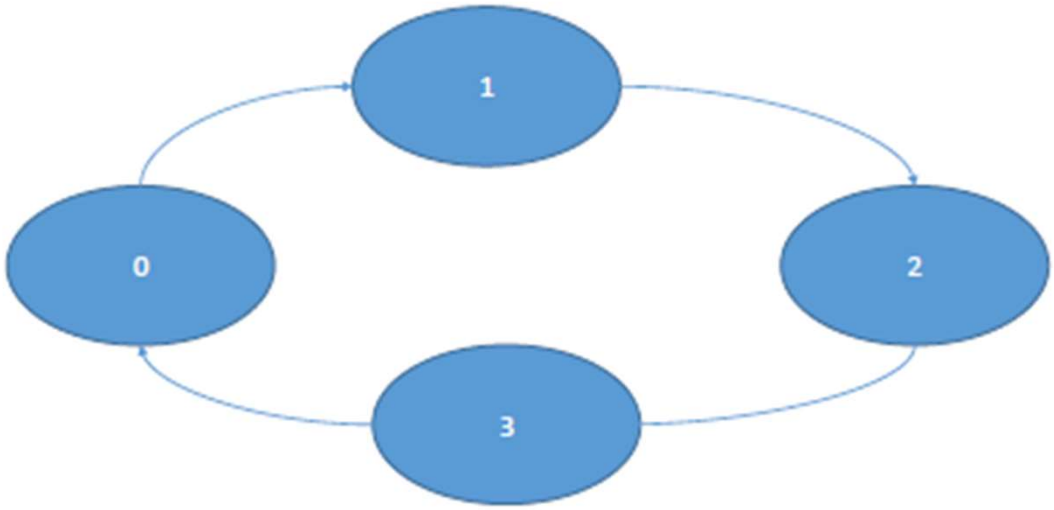
status.MPI_SOURCE
status.MPI_TAG

How much data am I receiving?

```
int MPI_Get_count(
      MPI_Status*   status_p  /* in  */,
      MPI_Datatype  type      /* in  */,
      int*          count_p   /* out */);
```



Round Robin Problem



round-robin-basic.c

```c
int
main (int argc, char **argv)
{
  int num_procs;
  int rank;

  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  printf("%d: hello (p=%d)\n", rank, num_procs);
  round_robin(rank, num_procs);
  printf("%d: goodbye\n", rank);

  MPI_Finalize();
}
```

round-robin-basic.c

```
void
round_robin(int rank, int procs)
{
  long int rand_mine, rand_prev;
  int rank_next = (rank + 1) % procs;
  int rank_prev = rank == 0 ? procs - 1 : rank - 1;
  MPI_Status status;

  srandom(time(NULL) + rank);
  rand_mine = random() / (RAND_MAX / 100);
  printf("%d: random is %ld\n", rank, rand_mine);
```

```
if (rank % 2 == 0) {
  printf("%d: sending %ld to %d\n", rank, rand_mine,
  ↪  rank_next);
  MPI_Send((void *)&rand_mine, 1, MPI_LONG, rank_next,
  ↪  1, MPI_COMM_WORLD);

  printf("%d: receiving from %d\n", rank, rank_prev);
  MPI_Recv((void *)&rand_prev, 1, MPI_LONG, rank_prev,
  ↪  1, MPI_COMM_WORLD, &status);

} else {
  printf("%d: receiving from %d\n", rank, rank_prev);
      MPI_Recv((void *)&rand_prev, 1, MPI_LONG,
      ↪  rank_prev, 1, MPI_COMM_WORLD, &status);

  printf("%d: sending %ld to %d\n", rank, rand_mine,
  ↪  rank_next);
  MPI_Send((void *)&rand_mine, 1, MPI_LONG, rank_next,
  ↪  1, MPI_COMM_WORLD);
}
```

```
  printf("%d: » I had %ld, %d had %ld\n",
         rank, rand_mine,
         rank_prev, rand_prev);
}
```

Output (as run)

```
1: hello (p=4)
1: random is 29
2: hello (p=4)
2: random is 65
2: sending 65 to 3
3: hello (p=4)
3: random is 51
3: sending 51 to 0
3: I had 51, 2 had 65
3: goodbye
```

```
0: hello (p=4)
0: random is 93
0: sending 93 to 1
0: I had 93, 3 had 51
0: goodbye
1: sending 29 to 2
1: I had 29, 0 had 93
1: goodbye
2: I had 65, 1 had 29
2: goodbye
```

Issues with send and receive

- Exact behavior is determined by the MPI implementation.
- MPI_Send may behave differently with regard to buffer size, cutoffs and blocking.
- MPI_Recv always blocks until a matching message is received.
- Know your implementation; don't make assumptions!



Deadlocks (Circular)

- Consider the following piece of code, in which process I sends a message to process i + 1 (modulo the number of processes) and receives a message from process i – 1 (modulo the number of processes).

```
1   int a[10], b[10], npes, myrank;
2   MPI_Status status;
3   ...
4   MPI_Comm_size(MPI_COMM_WORLD, &npes);
5   MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
6   MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1, MPI_COMM_WORLD);
7   MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1, MPI_COMM_WORLD);
8   ...
```

- The code is safe if buffering is used but will lead to unsafe state if MPI_SEND is blocking

Deadlocks→Solution

- We can break the circular wait to avoid deadlocks as follows:

```
1   int a[10], b[10], npes, myrank;
2   MPI_Status status;
3   ...
4   MPI_Comm_size(MPI_COMM_WORLD, &npes);
5   MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
6   if (myrank%2 == 1) {
7     MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1, MPI_COMM_WORLD);
8     MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1, MPI_COMM_WORLD);
9   }
10  else {
11    MPI_Recv(b, 10, MPI_INT, (myrank-1+npes)%npes, 1, MPI_COMM_WORLD);
12    MPI_Send(a, 10, MPI_INT, (myrank+1)%npes, 1, MPI_COMM_WORLD);
13  }
14  ...
```

Explanation

- This new implementation partitions the processes into two groups.
  - odd-numbered processes and
  - the even-numbered processes.

- The odd-numbered processes perform a send followed by a receive,
- and the even-numbered processes perform a receive followed by a send.

- Thus, when an odd-numbered process calls MPI_Send, the target process (which has an even number) will call MPI_Recv to receive that message, before attempting to send its own message.

Avoiding deadlocks using Simultaneous ***sendReceive*** operation

- To avoid earlier deadlocks, MPI provides **MPI_Sendrecv** function
  - It can both send and receive message
  - Does not suffers from the circular deadlock problems
  - One can think MPI_Sendrecv as allowing data to travel for both send and receive simultaneously.

```
int MPI_Sendrecv(void *sendbuf, int sendcount,
        MPI_Datatype senddatatype, int dest, int sendtag,
        void *recvbuf, int recvcount, MPI_Datatype recvdatatype,
        int source, int recvtag, MPI_Comm comm,
        MPI_Status *status)
```

- **MPI_Sendrecv_replace** function
  - If we wish to use the same buffer for both send and receive
  - First sends value[s] of current buffer and then overwrites them with received ones
- **Syntax**

```
int MPI_Sendrecv_replace(void *buf, int count,
        MPI_Datatype datatype, int dest, int sendtag,
        int source, int recvtag, MPI_Comm comm,
        MPI_Status *status)
```